Who Else Asked? Reuse and Leakage in Access-Aware KV-Cache Sharing for Multi-Tenant Retrieval-Augmented Generation

Anonymous ACL submission

Abstract

Retrieval-augmented generation (RAG) services prefill the same retrieved passages for many users, and prefix caching could reuse that computation. Because a shared cache leaks through response timing, serving engines isolate caches per user and forfeit reuse even among users who are entitled to read the same documents. We study *access-aware* KV-cache sharing, in which cache scopes are derived from the access-control lists (ACLs) of retrieved documents. We prove that ACL-derived scopes cost no reuse relative to unrestricted sharing, but we show that they open an insider channel that per-user isolation does not have: a co-authorized colleague who re-asks a guessed question learns from cache hits whether a colleague asked it, singling out an individual in 91.5% of probes and recovering the question text itself in 99.6% when queries inherit the scope of their evidence. We present ACLKV, which keeps queries in a private scope and promotes an evidence prefix to a shared scope only after k distinct authorized users have requested it. On retrieval-driven multi-tenant workloads built from HotpotQA and MuSiQue, ACLKV eliminates singling-out while retaining 87% of the reuse of unrestricted cache-aware sharing (4.8–8.5× that of per-user isolation); its advantage grows with the number of tenants and with shared context. A simulator validated against 192 live vLLM runs (mean error 0.055 points) also shows that workloads which inject popular documents synthetically overstate reuse by 6.6×.

1 Introduction

Retrieval-augmented generation (Lewis et al., 2020) conditions a language model on passages retrieved for each query. In enterprise deployments the retrieved evidence dominates the prompt: in our workloads a request carries 1393 (HotpotQA) to 2643 (MuSiQue) prompt tokens, almost all of them evidence, and every token must be prefilled before the first output token appears. Serving engines such as vLLM (Kwon et al., 2023) and SGLang (Zheng et al., 2024a) avoid recomputing a prompt prefix that an earlier request already processed by caching its key–value (KV) states in fixed-size blocks and reusing them on an exact prefix match (vLLM Project, 2026a).

Sharing those blocks across users is, however, a timing side channel: a cache hit shortens time-to-first-token (TTFT), so a user can test whether someone else recently sent a given prefix and, token by token, reconstruct other users' prompts (Wu et al., 2025; Zheng et al., 2024b; Song et al., 2024). Audits have found prompt caches shared across customers of commercial APIs (Gu et al., 2025). The standard mitigation is to isolate the cache per user or per request with a secret salt mixed into the first block hash (vLLM Project, 2026c; Addagada, 2026). Isolation is safe, but it also prevents two colleagues who read the *same* company handbook from sharing a single byte of KV state.

Enterprise RAG systems already know who may read each document: retrieval is filtered by document ACLs. This suggests deriving cache scopes from ACLs instead of from user identities, so that users who are jointly authorized for an evidence prefix share it while everyone else is isolated. Engines are beginning to support the required mechanism, several salt barriers inside one request (vLLM Project, 2025, 2026b). This paper asks two questions. *How much reuse does access-aware sharing recover under realistic multi-tenant RAG traffic?* And *is it actually safe?*

Our answers are mixed, and we believe that this is the useful result:

- **ACL scopes are free.** For traffic that passed the authorization filter, ACL-derived scopes produce exactly the cache hits of unrestricted sharing (Proposition 1); they cost nothing and

fully block clients that reach the engine without server-issued scope identifiers.

- **ACL scopes leak to insiders.** A co-authorized colleague can still learn about other users' requests. If the question inherits the scope of its evidence, as in a natural design (including our own first prototype), the colleague recovers the question text in 99.6% of probes. Even with private questions, the *sequence of retrieved passages* fingerprints the question: 91.5% of probes single out exactly one colleague who asked it. Per-user isolation has neither channel.

- **ACLKV closes the channel cheaply.** Keeping the question in a per-user scope costs at most 0.04 points of reuse, and k-*anonymous promotion*, which shares an evidence prefix only after k distinct authorized users requested it, reduces singling-out to 0% while retaining 87% of the reuse of unrestricted cache-aware sharing.

- **Realistic gains are modest at small budgets and grow with scale.** When overlap comes from a real retriever rather than from injected documents, access-aware sharing caches 5.42% of prompt tokens at a 2 GiB KV budget versus 1.13% for per-user isolation; the gap grows to 17.8× with 500 tenants, to 14.3% reuse at 16 GiB, and to 32.7% when tenants share pinned context.

We release the middleware, a 60-line vLLM plugin, a prefix-cache simulator validated against 192 live A100 runs, and the retrieval-driven multi-tenant workloads.[1]

2 Background and Threat Model

Prefix caching. vLLM stores KV states in blocks of B tokens (B=16). The hash of block j chains over the previous block, $h_j = H(h_{j-1}, \mathbf{t}_j, x_j)$, where $\mathbf{t}_j$ are the block's token ids and x_j optional extra keys. A request reuses its longest prefix of blocks whose hashes are cached; only full blocks are cached, and at least one token is always recomputed. A per-request `cache_salt` is mixed into x_1 only, so it isolates the entire request through the hash chain (vLLM Project, 2026c). A pending proposal generalises this to *salt regions*, i.e. (offset, salt) transitions within a request (vLLM Project, 2025, 2026b).

Threat model. The RAG middleware (retriever, authorization filter, prompt builder) and the serving engine are trusted; scope identifiers are derived with a secret server key. We consider two adversaries. A *bypass* client can submit raw token sequences to the engine (e.g., through a misconfigured internal endpoint) but cannot mint scope identifiers. An *insider* is an authenticated employee who uses the RAG front end as intended: they may ask any question, observe TTFT, and read every document their ACLs allow. We give the insider an upper-bound oracle: they learn exactly which blocks hit (in practice TTFT reveals this; Section 6.5) and their guessed question reproduces the victim's evidence. The insider's goal is to learn *whether, and which,* colleagues asked a question. Compromise of the middleware or key, leakage through generated answers, and physical side channels are out of scope.

3 Problem Formulation

Let U be users and G groups with membership $M \subseteq U \times G$. Every passage d carries an ACL $a(d)$, a conjunction of atoms `public`, `group:`g, or `user:`u, with audience $\mathrm{Aud}(a) \subseteq U$. A request $r = (u, q, D)$ consists of a user, a question, and retrieved passages $D = (d_1, \ldots, d_n)$ that passed the authorization filter, i.e. $u \in \mathrm{Aud}(a(d_i))$ for all i. The prompt is $\pi(r) = \sigma \,\|\, d_{o(1)} \|\, \cdots \|\, d_{o(n)} \,\|\, \tau(q)$ for a system preamble σ, an evidence order o, and a question template τ.

Because KV states depend on all preceding tokens, the state of an evidence prefix $P_k = \sigma\, d_{o(1)} \cdots d_{o(k)}$ inherits the restrictions of every passage in it. Its *effective ACL* is

$$\mathrm{ACL}(P_k) = \textstyle\bigwedge_{i \le k} a(d_{o(i)}), \tag{1}$$

$$\mathrm{Aud}(\mathrm{ACL}(P_k)) = \textstyle\bigcap_{i \le k} \mathrm{Aud}(a(d_{o(i)})). \tag{2}$$

A *scope assignment* ζ maps each prompt segment to a scope identifier that is mixed into the hash of the block where the scope begins; later blocks inherit it through the chain.

Definition 1 (Requirements)**.** *A scope assignment must satisfy, for every request by u that hits a block computed by another request:* ***(S1) authorization****: $u \in \mathrm{Aud}(\mathrm{ACL}(P))$ for the evidence prefix P that the block ends;* ***(S2) query privacy****: the block contains no question tokens of another*

[1] Code and data: https://anonymous.4open.science/r/aclkv (anonymized for review).

user; ***(S3) k-anonymity****: at least $k-1$ users other than u have requested the same evidence prefix. Moreover,* ***(S0) bypass resistance****: a request without server-issued scope identifiers hits no scoped block.*

The objective is to maximize the number of reused prompt tokens under a fixed KV-cache budget subject to S0–S3. Per-user isolation satisfies all requirements with almost no reuse; unrestricted sharing violates all of them.

Proposition 1 (ACL scopes are reuse-free)**.** *Let $\zeta_{\mathrm{ACL}}(P) = \mathrm{HMAC}_K(\mathrm{canon}(\mathrm{ACL}(P)))$ and suppose a passage's ACL is a function of its content. For every trace of requests that passed the authorization filter, the evidence blocks hit under ζ_{ACL} are exactly those hit under unrestricted sharing, and S0 and S1 hold.*

Proof sketch. If two prompts agree on their first j blocks, they contain the same passages in the same order up to block j, so their effective ACLs, and hence their scope identifiers, agree on those blocks; scoping therefore never separates two prompts that unrestricted sharing would let share. For S1, a hit by u on a block ending prefix P means u's own prompt contains P, every passage of which passed u's filter, so $u \in \mathrm{Aud}(\mathrm{ACL}(P))$. For S0, unscoped hashes differ from scoped ones on the first block of every scope. □

Proposition 1 has a sobering corollary: for honest traffic, the authorization filter alone already guarantees S1, and *all* of the privacy cost of access-aware sharing comes from S2 and S3, which ACL scopes do not provide. Our experiments confirm both halves (Sections 6.2 and 6.3).

4 AclKV

AclKV is middleware between retrieval and the serving engine (Figure 1).

ACL-derived scope barriers. For the ordered evidence, the planner computes $\mathrm{ACL}(P_k)$ incrementally and emits a barrier (offset, ζ) wherever it changes, with $\zeta = \mathrm{HMAC}_K(\mathrm{canon}(\mathrm{ACL}(P_k)))$ truncated to 64 bits. Identifiers are server-derived and fixed-size, the number of barriers is bounded, and the encoded list stays far below the engine's salt-length limit, which avoids a recently disclosed denial-of-service vector (vLLM Project, 2026d). We transport the barriers in vLLM's existing `cache_salt` field and add a 60-line general plugin that keys each scope into the block containing its offset; malformed salts fail closed to request-level isolation, and ordinary salts keep their stock semantics. The same barriers map one-to-one onto the proposed native `salt_regions` API (vLLM Project, 2026b). The middleware sends token ids rather than text, so barrier offsets are exact, and pads each segment to a block boundary so that a passage's blocks never depend on the length of its predecessor.

Algorithm 1 Scope plan with k-anonymous promotion

Require: user u; segments $s_0{=}\sigma, s_1, \ldots, s_{n+1}{=}\tau(q)$
Require: counter C; threshold k
$A \leftarrow$ `public`; *private* $\leftarrow$ **false**
for $i = 0$ **to** n **do**
  if $i \geq 1$ **then** $A \leftarrow A \wedge a(s_i)$
  $key \leftarrow (\mathrm{canon}(A), \mathrm{digest}(s_1..s_i))$
  if $\neg$*private* **and** $(|\mathrm{Aud}(A)| < k$ **or** $|C[key] \cup \{u\}| < k)$ **then**
    private $\leftarrow$ **true**
  end if
  $\zeta_i \leftarrow$ **if** *private* **then** $\mathrm{HMAC}_K(\texttt{user:}u)$ **else** $\mathrm{HMAC}_K(\mathrm{canon}(A))$
  $C[key] \leftarrow C[key] \cup \{u\}$
end for
$\zeta_{n+1} \leftarrow \mathrm{HMAC}_K(\texttt{user:}u)$ ▷ private query
return barriers where $\zeta_i \neq \zeta_{i-1}$

Private query scope. The question template $\tau(q)$, and therefore every generated token, is always placed in the requester's own scope $\mathrm{HMAC}_K(\texttt{user:}u)$. A user still reuses their own repeated questions, but no other user can hit them (S2).

k-anonymous promotion. Algorithm 1 enforces S3. The middleware counts, for each pair (effective ACL, evidence prefix), the distinct users who requested it. A segment uses its shared ACL scope only if the requester would be at least the k-th distinct requester and the prefix's audience has at least k members; otherwise that segment, and every later one, uses the requester's private scope. A cross-user hit therefore reveals only that at least $k-1$ other users asked for the same evidence (Sweeney, 2002). The counter is a bounded LRU map; an evicted prefix must be re-promoted, which fails closed.

Evidence order. Prefix reuse requires that related requests place shared evidence at the same positions. Following CacheWeaver (Tan et al., 2026), AclKV keeps a trie over a sliding win-

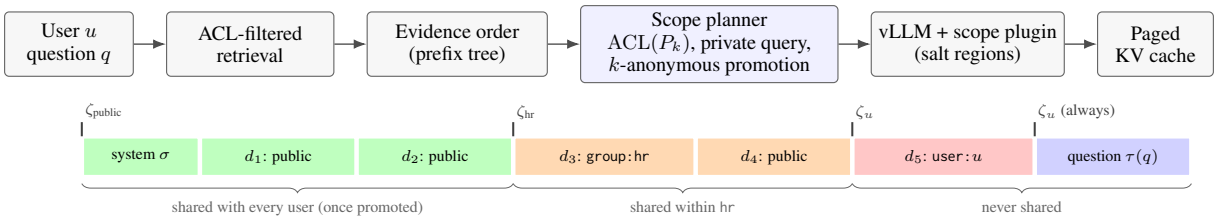


Figure 1: ACLKV. Top: the serving path; the only engine change is a block-hash extra key per scope barrier. Bottom: a prompt with its scope barriers (ζ). Each barrier is placed where the effective ACL of the evidence prefix narrows; later blocks inherit it through the hash chain, so the public passage d_4 placed after d_3 is shared only within hr. The question is always in the user's private scope, and a shared barrier takes effect only after k distinct authorized users requested that prefix.

dow of recently served evidence sequences and greedily places first the passages whose prefix was served most often, followed by the remaining passages in retrieval order. We evaluated two access-aware refinements, a walk that prefers already-promoted prefixes and a layout that places relevant evidence at the edges of the context; neither helped (Section 7.1), so ACLKV uses the tree walk, but enters only prefixes that at least k distinct users requested, so that the order of the passages reveals no more than a cache hit.

5 Experimental Setup

Data. We use the HotpotQA development set (Yang et al., 2018) (7,377 questions, 66,635 distinct passages) and MuSiQue-Ans (Trivedi et al., 2022) (2,417 questions, 21,100 passages). Passages are deduplicated by content; each request retrieves the top 10 (HotpotQA) or 20 (MuSiQue) passages from the *whole* corpus with BM25 (Robertson and Zaragoza, 2009), filtered by the requester's ACLs (gold-passage recall@k is 0.72 and 0.52). Conclusions are unchanged with dense (bge-small) or hybrid retrieval, which preserve the ranking of all policies with slightly lower absolute reuse (ACLKV: 4.73% and 5.11% vs. 5.42%; Appendix Table 8).

Multi-tenant workloads. We simulate an organization of 100 users in 10 departments nested in 3 divisions, plus 6 cross-functional groups with overlapping membership. Questions are clustered into 40 balanced topics (LSA k-means with capacity-constrained assignment); each topic is owned by a department. A passage is public with probability 0.5; otherwise it is restricted to its owning department, that department's division, or a cross-functional group (0.3), or is private to one department member (0.2). Users ask about their own department's topics with probability 0.8, and questions within a topic follow a Zipf distribution, so popular questions recur. To model paraphrases, 30% of a question's content terms are dropped *for retrieval only*; the prompt keeps the original question. Each workload has 5,000 requests with 1,605 (HotpotQA) and 1,046 (MuSiQue) distinct questions; 11% of requests repeat an evidence set exactly, yet 63% of their passages were retrieved before. Crucially, cross-request overlap is produced by the retriever, never injected. Variants change the number of tenants, ACL mix, paraphrase noise, and passages per request, and add 4 always-included public passages (*pinned*, e.g. a handbook). Appendix A gives details.

Policies. *No caching*; *per-user salt* (vLLM's mitigation and KVGov's per-principal salt; vLLM Project, 2026c; Addagada, 2026); *divergence salt* (shared system preamble, per-user afterwards; KVGov's divergence variant); *public/private split* (one boundary at the first non-public passage, the binary model of SafeKV with oracle labels; Chu et al., 2025); *ACL scopes* with retrieval order and with tree order; ACLKV (k=3); and two insecure upper bounds, *global* sharing with retrieval and with tree order (the latter is CacheWeaver as published). All secure policies use a private question scope unless stated otherwise.

Metrics. Efficiency: cached prompt tokens (%), recomputed tokens per request, evictions. Privacy: after 10% of requests a randomly chosen co-authorized insider re-issues the victim's question through the middleware (1,362 probes per policy on HotpotQA). A hit is *informative* if the block was computed only by other users; its *anonymity set* is the set of distinct other users who had requested that exact evidence prefix (or that exact question) at any earlier point. Using all history en-

larges the anonymity sets of the baselines and is therefore conservative for our defence. We report the fraction of probes that reveal anything, that *single out* one user (anonymity set of size one), that have an anonymity set smaller than 5, and that hit the question text. Bypass probes replay a victim's tokens without scope identifiers. Quality: EM and F1 against gold answers and aliases, with question-clustered bootstrap confidence intervals.

Platform and protocol. Large sweeps use a trace-driven simulator that mirrors vLLM 0.29's block manager (chained block hashes, the $n-1$ hit cap, LRU free queue, reverse-order release, touch-on-hit) with a closed loop of 16 concurrent requests, 48 output tokens, and Qwen2.5-7B-Instruct's KV geometry (Qwen Team, 2024) (56 KiB per token; a 2 GiB budget holds about 37k tokens). Live experiments run vLLM 0.29 on one NVIDIA A100-80GB with an explicit KV budget. All design decisions were made on HotpotQA seed 0; HotpotQA seeds 1–2 and all MuSiQue seeds are held out, and seed-0 values differ from held-out means by at most 0.10 points.

6 Results

6.1 Simulator fidelity and the latency model

We first validate the simulator against a pilot study of 192 live A100 runs (Appendix C): for every run we rebuilt the prompts locally (identical evidence order in all requests) and replayed them with the live completion lengths. Simulated and live cached-token fractions agree with a mean absolute error of 0.055 points (maximum 0.55, $r = 0.99997$; Appendix Figure 5). Live mean TTFT is a linear function of recomputed prompt tokens per request ($R^2 = 0.998$ at 16 concurrent requests, 0.92 ms per recomputed token; $R^2 = 0.984$ at 4). We therefore report cached tokens as the primary efficiency metric for large sweeps and confirm latency on the GPU (Section 6.5).

6.2 Main comparison

Table 1 summarizes the comparison. Three observations stand out. First, ACL scopes reproduce the reuse of global sharing exactly, as Proposition 1 predicts (5.20% vs. 5.20%), and cache-aware tree ordering adds 1.05 points on top of either. Second, absolute reuse is modest at this budget: the system preamble alone (divergence salt) accounts for 4.67% of prompt tokens, and sharing evidence across users adds at most another 1.6 points on HotpotQA and 1.4 on MuSiQue, because a 2 GiB cache holds only about 27 requests while identical questions recur a median of 284 (HotpotQA) and 229 (MuSiQue) requests apart. Third, the picture changes completely when tenants share context: with four pinned public passages, scoped sharing caches 32.5% of prompt tokens versus 5.7% for per-user and 8.5% for divergence salting. ACLKV gives up a little reuse for k-anonymity (5.42% vs. 6.25%) and remains 4.8× (HotpotQA) and 8.5× (MuSiQue) more effective than per-user isolation.

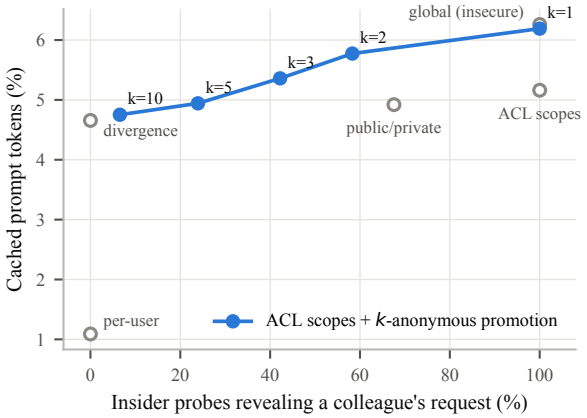


Figure 2: Privacy–utility frontier on HotpotQA (2 GiB, three seeds). The blue curve varies the promotion threshold k of ACLKV; hollow points are baselines.

6.3 Insider leakage and the privacy–utility frontier

Table 2 shows that access-aware sharing is not leak-free. Under ACL scopes, every insider probe reveals that a colleague recently retrieved the same evidence, and 91.5% (MuSiQue: 89.6%) single out exactly one colleague; when the question inherits its evidence scope, the insider additionally recovers the question text in 99.6% of probes. The public/private split leaks less (49.2%) only because it shares less. Per-user and divergence salting reveal nothing.

k-anonymous promotion closes the singling-out channel: with k=3, no probe singles out a colleague (smallest anonymity set observed: 2, as guaranteed), only 42.2% reveal anything, and 28.7% have an anonymity set below five. Figure 2 traces the frontier: each increase of k trades a few tenths of a point of reuse for a large drop in what an insider learns; at k=10 only 6.5% of probes reveal anything, and reuse still exceeds divergence salting.

Policy	Isolation	Cached prompt tokens (%) HotpotQA	MuSiQue	Pinned	vs. per-user	Insider singled-out (%)
No caching	full	0.00±0.00	0.00±0.00	0.0	–	0.0
Per-user salt	per user	1.13±0.01	0.38±0.02	5.7	1.0×	0.0
Divergence salt	per user†	4.67±0.03	2.47±0.03	8.5	4.1×	0.0
Public/private split	public/user	4.95±0.02	2.69±0.02	32.3	4.4×	49.2
ACL scopes	ACL	5.20±0.02	2.84±0.05	32.5	4.6×	91.5
ACL scopes + tree order	ACL	6.25±0.12	3.84±0.15	33.2	5.5×	93.7
ACLKV (k=3)	ACL + k-anon.	**5.42**±0.06	**3.23**±0.13	32.7	4.8×	0.0
Global sharing	none	5.20±0.02	2.84±0.05	32.5	4.6×	91.5
Global + tree order	none	6.25±0.12	3.84±0.15	33.2	5.5×	93.7

Table 1: Cached prompt tokens at a 2 GiB KV budget (mean ± s.d. over three workload seeds; *pinned*: one seed), relative reuse versus per-user isolation on HotpotQA, and the fraction of insider probes that single out one colleague. †Shared system preamble. Global sharing is insecure and shown as an upper bound; it also leaks to bypass clients (60,876 honest blocks hit), unlike every scoped policy (0).

Policy	HotpotQA insider probes (%) reveal	single	anon<5	query	MuSiQue single (%)	Cached (%)
Global sharing (insecure)	100.0	91.5	99.9	99.6	90.6	5.20
Per-user salt	0.0	0.0	0.0	0.0	0.0	1.09
Divergence salt	0.0	0.0	0.0	0.0	0.0	4.66
Public/private split	67.6	49.2	63.1	0.0	44.8	4.92
ACL scopes, inherited query (v1)	100.0	91.5	99.9	99.6	90.6	5.16
ACL scopes	100.0	91.5	99.9	0.0	89.6	5.16
ACL scopes + tree order	100.0	93.7	99.9	0.0	95.2	6.19
ACLKV, k=2	58.3	2.4	51.2	0.0	2.5	5.77
ACLKV, k=3	42.2	**0.0**	28.7	0.0	0.0	5.36
ACLKV, k=5	23.9	0.0	2.6	0.0	0.0	4.94
ACLKV, k=10	6.5	0.0	0.0	0.0	0.0	4.75

Table 2: Insider probes through the middleware (HotpotQA: three seeds; MuSiQue: one seed; 2 GiB). *reveal*: any informative hit; *single*: anonymity set of one; *query*: hit on the victim's question text. Cached: honest requests in the same trace.

Policy	Cached (%)	TTFT p50 (ms)	TTFT p95 (ms)	req/s	EM / F1
Per-user salt	TBD	TBD	TBD	TBD	TBD
Divergence salt	TBD	TBD	TBD	TBD	TBD
ACL scopes	TBD	TBD	TBD	TBD	TBD
ACL scopes + tree order	TBD	TBD	TBD	TBD	TBD
ACLKV (k=3)	TBD	TBD	TBD	TBD	TBD
Global (insecure)	TBD	TBD	TBD	TBD	TBD

Table 3: Live vLLM 0.29 on one A100 (Qwen2.5-7B-Instruct, 2 GiB KV budget, HotpotQA workloads, 2,000 requests, 16 concurrent). **TBD**: pending runs (`scripts/sol_array.sbatch`; Qwen3-8B, Llama-3.1-8B and Qwen2.5-32B and open-loop goodput in Appendix D).

6.4 When does access-aware sharing pay off?

Figure 3 varies the deployment. *Tenants:* per-user isolation collapses as the population grows (3.58% with 20 users, 0.30% with 500), whereas ACL scopes are flat (5.23–5.33%), so the advantage grows from 1.5× to 17.8×. *Budget:* reuse grows with the KV budget for every policy, fastest for tree-ordered sharing (5.4% at 1 GiB to 14.3% at 16 GiB). The cost of k-anonymity grows with the budget too: ACLKV retains 59% of tree-ordered reuse at 16 GiB, because a larger cache keeps more question-specific prefixes that never reach k requesters. *Paraphrase noise* reduces reuse for all policies; the gain of tree ordering over retrieval order is largest when evidence sets overlap but are reordered (1.27 points at 15% noise versus 0.90 for exact queries and 0.66 at 50%). Appendix Table 6 adds prompt length, model KV geometry, ACL mix, and concurrency: longer prompts and larger per-token KV footprints reduce reuse (Qwen2.5-32B fits 4.6× fewer tokens per GiB than Qwen2.5-7B), and a more restrictive ACL mix slightly *increases* reuse (7.4% vs. 6.2% with tree order), plausibly because ACL filtering concentrates each department's retrieval on its own passages.

6.5 Live serving

Table 3 will report TTFT, throughput, and answer quality measured end-to-end on the GPU for the realistic workloads. *[Pending A100 runs.]* The pilot study already establishes the mechanism on real hardware: with the synthetic-injection workload at 50% overlap (Appendix C), cache-aware sharing cut median TTFT from 1461 ms (per-user) to 743 ms and raised throughput from 8.5 to 13.7 requests/s, and the live bypass probe confirmed the timing channel: unscoped replays of a victim's prompt returned in 19 ms versus 128 ms cold under global sharing, and hit nothing under ACL scopes (137 ms). Given the TTFT model of Section 6.1, we expect the realistic workloads' smaller reuse to translate into proportionally smaller latency gains at 2 GiB and larger ones at 8 GiB.

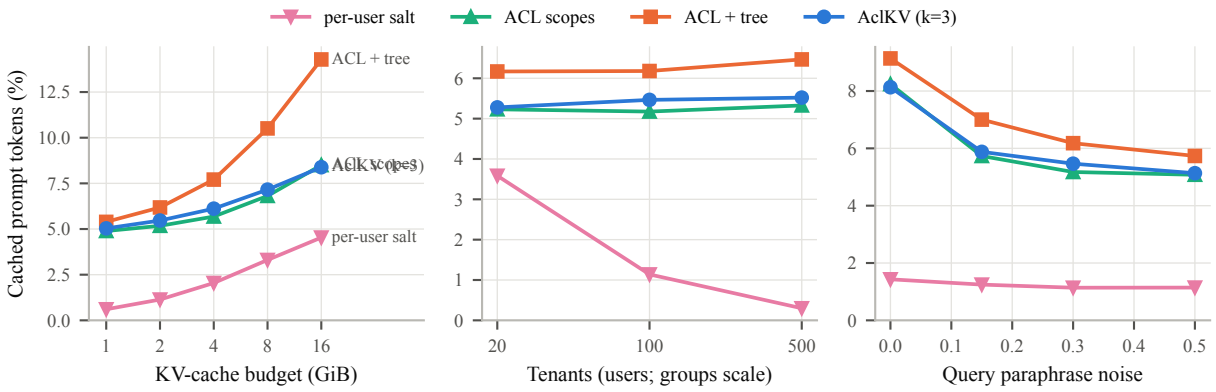


Figure 3: Cached prompt tokens on HotpotQA as a function of the KV budget, the number of tenants, and query paraphrase noise. Global sharing with tree order coincides with *ACL + tree* in every setting (Proposition 1).

Evidence order	Cached (%)	Gold mid (%)	Top-1 at edge (%)
Retrieval order	4.59	31.4	100.0
Broad-to-narrow (v1 B4)	4.62	33.9	76.1
Popularity-canonical (v1 B5)	4.47	36.6	67.8
+ relevance anchor (v1 B6)	4.32	22.9	100.0
LongContextReorder	4.32	19.7	100.0
Edge layout	4.89	31.7	93.8
Prefix tree (CacheWeaver-style)	5.69	32.6	76.6
Prefix tree, k=3	5.18	32.6	76.6
Privacy-aware tree, k=3	5.17	32.6	76.6

Table 4: Evidence ordering under ACL scopes (mean over three HotpotQA seeds and one MuSiQue seed, 2 GiB). *Gold mid*: answer-bearing passages in positions 3 to $n-2$; *Top-1 at edge*: highest-ranked passage first or last.

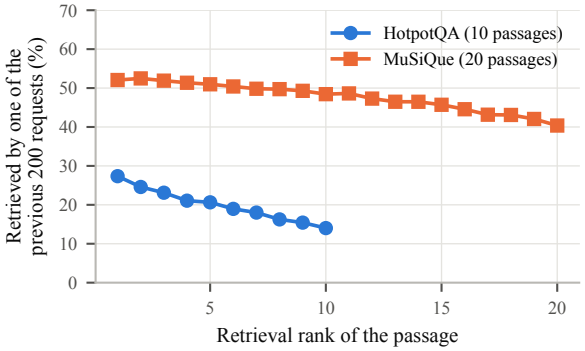


Figure 4: Probability that a retrieved passage was retrieved by one of the previous 200 requests, by retrieval rank (three seeds each). Highly relevant passages are the most reusable.

7 Analysis and Ablations

7.1 Evidence order: relevance and reuse are aligned

Table 4 compares orderings. The CacheWeaver-style tree yields the most reuse (5.69% vs. 4.59% for retrieval order). Two plausible access-aware refinements fail. Our pilot proposed placing the most relevant passages next to the question, motivated by the observation that language models attend most to the edges of a long context (Liu et al., 2024) and that, under synthetic injection, relevant passages were question-specific and shared ones were not. With real retrieval this premise is false: highly ranked passages are the *most* reusable ones (Figure 4; on HotpotQA, 27.4% of rank-1 passages were retrieved by one of the previous 200 requests versus 14.0% of rank-10 passages), so moving them to the end of the prompt shortens shared prefixes (4.89%; LongContextReorder (LangChain, 2023): 4.32%). Reuse and relevance therefore both favour the *front* of the context. Second, a privacy-aware walk that prefers already-promoted prefixes is indistinguishable from the plain tree under k=3 (5.17% vs. 5.18%). The tree moves the top-ranked passage away from the first position in a quarter of requests (top-1 at an edge: 76.6%), which is why we measure answer quality directly (Section 7.4).

7.2 Mechanism ablations

Private query scope costs no measurable reuse on the held-out workloads (below 0.01 points) and 0.04 points even when questions repeat verbatim (exact-query workload), while removing all question-text leakage (Table 2). *Block alignment* matters: without padding segments to block boundaries, ACLKV caches 4.56% instead of 5.47% (padding adds 5.5% tokens). The tree window is insensitive between 64 and 4,096 sequences (6.16–6.22%).

7.3 Position-independent caching

A natural objection to prefix caching is that the same passage appears in different contexts, which changes the token prefix and prevents reuse.

Overlap	ACL-scope F1	ΔF1 v1-B5	ΔF1 v1-B6 [95% CI]	p
0.00	0.685	-0.004	+0.017 [-0.023, +0.056]	0.42
0.25	0.681	-0.002	+0.023 [-0.023, +0.074]	0.34
0.50	0.649	-0.002	+0.029 [-0.015, +0.074]	0.20
0.75	0.679	+0.007	+0.013 [-0.025, +0.052]	0.50

Table 5: Pilot answer quality on the live A100 (HotpotQA, Qwen2.5-7B-Instruct, 400 requests per overlap level). Differences to retrieval order; 95% question-clustered bootstrap CIs.

Position-independent caching addresses this directly by caching each passage's KV separately and recomputing a fraction r of tokens to restore cross-passage attention (Yao et al., 2025). In our simulator (Appendix Table 7), it raises reuse from 6.18% to 7.47% on HotpotQA and from 4.00% to 8.12% on MuSiQue at r=0.15. Two findings transfer from Section 3: because a separately encoded passage depends only on itself, its correct scope is its *own* ACL rather than the prefix's, and own-ACL scoping again costs at most 0.01 points relative to unscoped chunk caching; without any scope, chunk caching produced 25 unauthorized hits on MuSiQue from honest traffic alone, because passages with different ACLs can share identical leading text. Position-independent reuse is complementary to ACLKV but requires engine changes and has its own timing channel (Sun et al., 2026).

7.4 Answer quality

Cache-aware ordering changes what the model reads first, so it could affect answers. In the pilot (Table 5), neither popularity-canonical ordering nor relevance anchoring changed F1 significantly at any overlap level (all 95% intervals contain zero; range -0.025 to +0.074). Requests with identical token sequences produced different answers under different salting policies in at most 6 of 400 cases (numerical differences between cached and recomputed prefill), which bounds the effect of salting itself. The corresponding comparison for tree ordering on the realistic workloads, with an LLM judge in addition to EM/F1, is part of the pending GPU runs (**TBD**).

7.5 Synthetic overlap overstates reuse

Our pilot followed common practice and created cross-user overlap by injecting popular passages into requests. Under that workload, cache-aware sharing reached 41.2% cached tokens live, 6.6× the 6.25% it reaches when overlap comes from retrieval. Injected passages are irrelevant to the question, rank low, and can be moved freely to the front; real shared passages are relevant, rank high, and already sit near the front. We recommend that evaluations of RAG caching derive overlap from a retriever and report it (Section 5).

8 Related Work

Prefix and RAG caching. PagedAttention (Kwon et al., 2023) and RadixAttention (Zheng et al., 2024a) reuse exact prompt prefixes; Prompt Cache (Gim et al., 2024) reuses modular segments. RAG-specific systems cache knowledge across requests (Jin et al., 2024), precompute or fuse per-passage KV states (Yao et al., 2025; Lu et al., 2024; Sun et al., 2024), or reorder evidence to create shared prefixes (Tan et al., 2026). None of these considers which users may share a cached state; we compose with CacheWeaver-style ordering and show that access control does not change the best order.

KV-cache side channels and defences. Timing attacks reconstruct prompts from shared prefix caches (Wu et al., 2025; Zheng et al., 2024b; Song et al., 2024), including non-prefix RAG caches (Sun et al., 2026), and audits found shared caches in commercial APIs (Gu et al., 2025; TODO, 2026a). Defences isolate per user or principal (vLLM Project, 2026c; Addagada, 2026), share only content classified as non-sensitive (Chu et al., 2025; Wu et al., 2026), tag cache entries with their owner and isolate prefixes whose reuse looks suspicious (Pennas et al., 2026), protect admission (TODO, 2026b), or gate sharing by organizational policy (TODO, 2026c,d). vLLM's salt-region proposal provides the multi-barrier mechanism (vLLM Project, 2025, 2026b); we do not claim it as a contribution. Our contributions are deriving barriers from authoritative document ACLs, the reuse-freeness result, the insider analysis showing that authorized sharing still leaks retrieval fingerprints, and k-anonymous promotion as a defence.

Access control in RAG. Enterprise RAG systems filter retrieval by document permissions (TODO, 2025); we assume such a filter and study what it implies for the serving layer.

9 Conclusion

Deriving KV-cache scopes from document ACLs is free: it reproduces the reuse of unrestricted sharing and fully stops clients without scope identifiers. It is not, however, private: authorized sharing lets colleagues learn about each other's questions through retrieval fingerprints. Private query scopes and k-anonymous promotion close that channel at a small cost. Under realistic traffic the efficiency case for cross-user evidence sharing is strongest with many tenants, large caches, and shared context, and evaluations should derive overlap from retrieval rather than inject it.

Limitations

Our efficiency sweeps use a simulator; it reproduces live vLLM cache behaviour closely (0.055-point error), but it does not model scheduling time, so the effect of concurrency and arrival rate on latency rests on the pending open-loop GPU runs. The workloads are synthetic in their organizational structure: ACLs, group memberships, and topic ownership are assigned by a generative model over public QA corpora, and paraphrases are approximated by dropping query terms for retrieval; real enterprise logs may show different overlap. The insider is granted an oracle of block-level hits and a perfect question guess, which upper-bounds leakage; real TTFT measurements are noisier. k-anonymous promotion assumes that identities are bound to enterprise accounts; an attacker who controls $k-1$ accounts defeats it (Douceur, 2002), and it does not hide *that* a popular prefix exists. The anonymity set counts requesters over the whole trace, which is conservative for comparisons but does not equal the information revealed by a single timing observation. We evaluate one 7B model in simulation geometry and live; larger models and other engines remain to be measured. Finally, answer-quality results for the realistic workloads are pending (Table 3).

Ethics Statement

This work studies a privacy risk in shared LLM serving infrastructure and proposes a mitigation. The attacks we evaluate are simulated against synthetic organizations built from public QA datasets; no real user data or third-party systems were probed. We describe the insider channel at the level needed for defenders and do not release attack tooling beyond the simulator's measurement harness.

References

Tejasvi C. Addagada. 2026. Governing the KV cache: Preventing timing side-channel leakage in multi-tenant LLM inference. *arXiv preprint arXiv:2608.09225*.

Kexin Chu et al. 2025. Selective KV-cache sharing to mitigate timing side-channels in LLM inference. *arXiv preprint arXiv:2508.08438*.

Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. 2009. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759.

John R. Douceur. 2002. The Sybil attack. In *Peer-to-Peer Systems: First International Workshop (IPTPS)*, pages 251–260.

In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems (MLSys)*.

Chenchen Gu, Xiang Lisa Li, Rohith Kuditipudi, Percy Liang, and Tatsunori Hashimoto. 2025. Auditing prompt caching in language model APIs. In *Proceedings of the International Conference on Machine Learning (ICML)*.

Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Xin Liu, Xuanzhe Liu, and Xin Jin. 2024. RAGCache: Efficient knowledge caching for retrieval-augmented generation. *arXiv preprint arXiv:2404.12457*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*.

LangChain. 2023. LongContextReorder document transformer. `langchain_community.document_transformers`.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

Songshuo Lu, Hua Wang, Yutian Rong, Zhi Chen, and Yaohua Tang. 2024. TurboRAG: Accelerating retrieval-augmented generation with precomputed KV caches for chunked text. *arXiv preprint arXiv:2410.07590*.

Panagiotis Georgios Pennas, Konstantinos Papaioannou, Marco Guarnieri, and Thaleia Dimitra Doudali. 2026. PrefixWall: Mitigating prefix caching side channels in shared LLM systems. *arXiv preprint arXiv:2603.10726*.

Qwen Team. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

Linke Song, Zixuan Pang, Wenhao Wang, Zihao Wang, XiaoFeng Wang, Hongbo Chen, Wei Song, Yier Jin, Dan Meng, and Rui Hou. 2024. The early bird catches the leak: Unveiling timing side channels in LLM serving systems. *arXiv preprint arXiv:2409.20002*.

East Sun, Yan Wang, and Lan Tian. 2024. Block-attention for efficient prefilling. *arXiv preprint arXiv:2409.15355*.

He Sun, Shinan Liu, Siyuan Ma, Junhao Li, Mingjun Xiao, and Wenhao Jiang. 2026. Agent-assisted side-channel attacks on non-prefix KV cache in RAG. *arXiv preprint arXiv:2606.21842*.

Latanya Sweeney. 2002. k-anonymity: A model for protecting privacy. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 10(5):557–570.

Kaizhen Tan, Rong Gu, and Mingyuan Li. 2026. CacheWeaver: Cache-aware evidence ordering for efficient grounded RAG inference. *arXiv preprint arXiv:2606.19667*.

TODO. 2025. Todo: permission-aware / access-controlled retrieval for enterprise RAG. *TODO*.

TODO. 2026a. KeyPooling: Measuring where LLM API relay paths collapse prompt cache isolation. *arXiv preprint arXiv:2608.17485*.

TODO. 2026b. Preserving admission responsibility in multi-tenant large language model prefix caches. *arXiv preprint arXiv:2608.01657*.

TODO. 2026c. Separating intelligence from inference: A standard for edge-native AI computing. *arXiv preprint arXiv:2608.02608*.

TODO. 2026d. Unveiling and mitigating privacy risks of KV-cache in LLM inference. In *Network and Distributed System Security Symposium (NDSS)*.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554.

vLLM Project. 2025. RFC: Cache salting for secure and flexible prefix caching in vLLM. GitHub issue #16016.

vLLM Project. 2026a. Automatic prefix caching. vLLM documentation, v0.29.0.

vLLM Project. 2026b. Per-token-range cache salts (`salt_regions`). GitHub pull request #52655 (open as of October 2026).

vLLM Project. 2026c. Security: Prefix cache timing side-channel mitigation (cache salting). vLLM documentation, v0.29.0.

vLLM Project. 2026d. vLLM DoS via unbounded `cache_salt` length. GitHub Security Advisory GHSA-wpww-v874-ph2p.

Guanlong Wu, Zhaohan Li, Yao Zhang, Zheng Zhang, Jianyu Niu, Ye Wu, and Yinqian Zhang. 2026. CachePrune: Privacy-aware and fine-grained KV cache sharing for efficient LLM inference. *arXiv preprint arXiv:2605.23640*.

Guanlong Wu, Zheng Zhang, Yao Zhang, Weili Wang, Jianyu Niu, Ye Wu, and Yinqian Zhang. 2025. I know what you asked: Prompt leakage via KV-cache sharing in multi-tenant LLM serving. In *Network and Distributed System Security Symposium (NDSS)*.

Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. C-Pack: Packaged resources to advance general Chinese embedding. *arXiv preprint arXiv:2309.07597*.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380.

Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the European Conference on Computer Systems (EuroSys)*.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez,

Clark Barrett, and Ying Sheng. 2024a. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Xinyao Zheng, Husheng Han, Shangyi Shi, Qiyan Fang, Zidong Du, Xing Hu, and Qi Guo. 2024b. InputSnatch: Stealing input in LLM services via timing side-channel attacks. *arXiv preprint arXiv:2411.18191*.

A Workload Generation

Organization. Users are assigned round-robin to departments; departments are assigned round-robin to divisions; each user also joins one random cross-functional group. **Topics.** Questions are embedded with TF-IDF and truncated SVD (100 dimensions); k-means centroids are computed and questions are assigned greedily to the nearest centroid with capacity $\lceil N/40 \rceil$, which avoids the degenerate clusters of plain k-means on short questions (which in our development runs made some users repeat a handful of questions). **ACLs.** A passage's owning topic is that of the first question whose context contains it; group-restricted passages go to the owning department (60%), its division (25%), or a random cross-functional group (15%). **Requests.** A user is drawn uniformly; a topic is drawn from the user's department with probability 0.8; a question is drawn by Zipf (s=1) within the topic and re-drawn (up to 25 times) until the user can read its gold passages. Retrieval is ACL-filtered BM25 top-k over the full corpus (dense and hybrid retrieval in Appendix B). **Seeds.** Each workload is generated with seeds 0, 1, 2.

B Additional Results

Setting	Per-user	ACL scopes	ACL + tree	ACLKV	Global + tree
20 users	3.58	5.23	6.17	5.28	6.17
100 users	1.14	5.17	6.18	5.47	6.18
500 users	0.30	5.33	6.47	5.52	6.47
10 docs, 8 GiB	3.31	6.81	10.51	7.16	10.52
20 docs, 8 GiB	1.07	2.96	4.59	3.19	4.59
40 docs, 8 GiB	0.32	1.34	1.94	1.44	1.94
1 GiB	0.61	4.89	5.39	5.04	5.39
4 GiB	2.05	5.68	7.71	6.12	7.71
16 GiB	4.53	8.50	14.28	8.38	14.31
Qwen2.5-7B-Instruct (8 GiB)	3.31	6.81	10.51	7.16	10.52
Qwen3-8B (8 GiB)	1.68	5.44	7.07	5.87	7.08
Qwen2.5-32B-Instruct (8 GiB)	0.97	5.06	5.92	5.32	5.92
ACL mix 0.2/0.4/0.4	1.23	5.62	7.37	6.10	7.37
ACL mix 0.8/0.15/0.05	1.09	5.16	6.00	5.16	6.00
concurrency 1	1.15	5.18	6.19	5.47	6.19
concurrency 32	1.13	5.17	6.17	5.47	6.17

Table 6: Cached prompt tokens (%) on HotpotQA (seed 0) under scaling variants; 2 GiB unless stated.

Reuse mechanism	HotpotQA	MuSiQue
Prefix, ACL scopes + tree order	6.18	4.00
Position-indep., own-ACL scope, r=0.15	7.47	8.12
Position-indep., own-ACL scope, r=0.3	6.97	7.12
Position-indep., per-user scope, r=0.15	4.86	2.89
Position-indep., per-user scope, r=0.3	4.82	2.81
Position-indep., no scope (insecure), r=0.15	7.47	8.13
Position-indep., no scope (insecure), r=0.3	6.97	7.12

Table 7: Prefix caching vs. position-independent chunk caching (seed 0, 2 GiB); r is the recomputed fraction of each reused chunk.

Policy	BM25	Dense	Hybrid
Per-user salt	1.13	0.94	1.05
Divergence salt	4.67	4.22	4.47
ACL scopes	5.20	4.63	4.95
ACL scopes + tree order	6.25	5.43	5.93
ACLKV (k=3)	5.42	4.73	5.11
Global + tree order	6.25	5.43	5.93

Table 8: Cached prompt tokens (%) with BM25, dense (bge-small; Xiao et al., 2023) and hybrid (reciprocal-rank fusion; Cormack et al., 2009) retrieval (HotpotQA, three seeds, 2 GiB).

C Pilot Study

The pilot used the original synthetic workload: 400 requests from 20 users over HotpotQA, where a fraction (0–75%) of each request's distractor passages was replaced by passages from a 64-element Zipf-popular pool, together with eight policies (including a design in which the question inherited its evidence scope), three KV budgets, and two concurrency levels, for 192 runs of vLLM 0.29 with Qwen2.5-7B-Instruct on one A100. It is the source of the fidelity and latency analyses (Figure 5), the quality comparison (Table 5), and the overstatement measurement (Section 7.5).

D Reproducibility

All CPU experiments: `python scripts/run_v2.py` (about 15 minutes on 10 cores). GPU matrix (4 models × {2, 8} GiB, closed loop, open-loop Poisson rates 4–16 req/s with a 1 s TTFT SLO, quality runs with an LLM judge, live insider probe): `sbatch --array=0-7 scripts/sol_array.sbatch`, then `python scripts/analyze_live_v2.py` and `python scripts/make_paper_assets.py`, which regenerates every table and number in this paper.

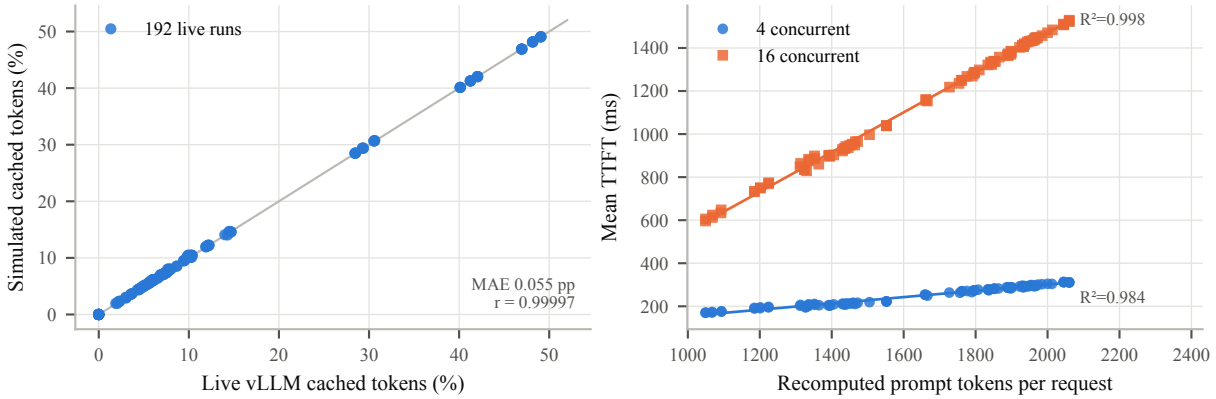


Figure 5: Left: simulated vs. live vLLM cached-token fraction for 192 A100 runs. Right: live mean TTFT is linear in the number of recomputed prompt tokens per request (Qwen2.5-7B-Instruct, 2 GiB, closed loop).